



UPPSALA
UNIVERSITET

IT 22 004

Examensarbete 30 hp
Januari 2022

Implementation of the signature scheme Rainbow on SoC FPGA

Tomas Preucil

Institutionen för informationsteknologi
Department of Information Technology



UPPSALA
UNIVERSITET

**Teknisk- naturvetenskaplig fakultet
UTH-enheten**

Besöksadress:
Ångströmlaboratoriet
Lägerhyddsvägen 1
Hus 4, Plan 0

Postadress:
Box 536
751 21 Uppsala

Telefon:
018 – 471 30 03

Telefax:
018 – 471 30 00

Hemsida:
<http://www.teknat.uu.se/student>

Abstract

Implementation of the signature scheme Rainbow on SoC FPGA

Tomas Preucil

Thanks to the research progress, quantum computers are slowly becoming a reality, and some companies already have working prototypes. While this is great news for some, it also means that part of the encryption used today will be rendered unsafe and obsolete. Due to this fact, NIST (US National Institute of Standards and Technology) has been running a standardisation process for quantum-resistant key exchange algorithms and digital signatures. One of these is Rainbow - a signature scheme based on the fact that solving a set of random multivariate quadratic equations is an NP-hard problem.

This thesis aims to develop an AXI connected accelerator for the Rainbow signature scheme, specifically the la variant. The accelerator is highly parameterisable, so one can choose the data bus width, directly affecting the FPGA area used. It is also possible to swap components to use the design for other variants of Rainbow. Thanks to that, multiple tests can be performed.

The developed accelerator provides significant speedup compared to CPU-based computation. This thesis includes detailed documentation of the design as well as performance and resource utilisation tests.

Handledare: Petr Socha
Ämnesgranskare: Christian Rohner
Examinator: Philipp Rummer
IT 22 004
Tryckt av: Reprocentralen ITC

Acknowledgements

I would like to thank my supervisor Petr Socha for his help. He was always available and he never got angry at me even though I probably deserved it more than once. Next I would like to thank my family for their psychical and financial help. I would also like to thank my friends from GH nation. I could not do it without you. Lastly, thanks to all the cats that would let me pet or hold them – you saved my sanity.

Uppsala University
Faculty of Science and Technology

Citation of this thesis

Přeučil, Tomáš. *Implementation of the signature scheme Rainbow on SoC FPGA*. Master's thesis. Uppsala University, Faculty of Science and Technology, 2022.

Contents

1	Introduction	1
1.1	Goals	2
2	Theoretical background	3
2.1	Quantum computing	4
2.2	Need for post-quantum cryptography	4
2.2.1	Shor’s algorithm	4
2.3	NIST Post-Quantum Cryptography Standardization Process . .	5
2.4	Algorithm types	6
2.5	Rainbow	7
2.5.1	Basic idea	7
2.5.2	Key structure for Rainbow	8
2.5.3	Security levels	8
2.5.4	Parameters for Rainbow	9
2.5.5	Algorithm as presented by its authors	9
3	System overview	11
3.1	Algorithm description	11
3.2	Design	12
3.2.1	Linear equation solver	12
3.2.1.1	Overview of the solver’s design	13
3.2.1.2	Basic cell	14
3.2.1.3	Pivot cell	15
3.2.1.4	Pivot column cell	15
3.2.1.5	Pivot Row cell	15
3.2.2	Modification of the GSMITH algorithm	16
3.2.2.1	Registers	16
3.2.2.2	Wrappers	17
3.2.2.3	Modification for inversion	17

3.3	Communication	18
3.4	Related research	18
3.4.1	Approach of S. Balasubramanian et al.	18
3.4.2	Approach of H. Yi and Z. Nie	19
3.4.3	Approach of A. Ferozpuri and K. Gaj	19
4	Accelerator design	21
4.1	Block diagram	21
4.2	Parallelisation	22
4.3	Communication	23
4.4	Chosen variant	23
4.5	Parametrisation	23
4.6	Linear equation solver	24
4.7	Separation to modules	24
5	Implementation	25
5.1	Platform and language choice	25
5.2	Top-level memory implementation	25
5.3	RAM/ROM structure	26
5.3.1	Key memory	26
5.3.2	Temporary memory	26
5.4	Implementation of modules	27
5.4.1	Registers	27
5.4.1.1	4096-bit register	27
5.4.1.2	128-bit register	27
5.4.1.3	144-bit register	27
5.4.2	GF16 Multiplication	28
5.4.3	Matrix multiplication	28
5.4.4	“Quadratic” triangular matrix evaluation	28
5.4.5	Adder	29
5.4.6	Memory reset	29
5.4.7	Memory copy	29
5.4.8	Controller	30
5.4.9	GSMITH	30
5.5	Top-level connection and interface	30
5.6	ARM implementation	30
5.7	Input and output	30
6	Testing and evaluation	35
6.1	Testing	35
6.1.1	Reference implementation in ARM	35
6.1.2	Memory access	35
6.1.3	Linear equation solver	36
6.1.3.1	Modification of GSMITH for inversion	36

6.1.4	Other submodules	36
6.1.5	Top-level module	36
6.1.6	Different parameters	37
6.2	Evaluation	37
6.2.1	Testing in the ARM processor	37
6.2.2	Used area	37
6.2.3	Number of clock cycles	38
6.2.4	Maximum possible frequency and total computation time	38
6.2.5	Power consumption	38
6.2.6	Summary	39
Conclusion		41
Bibliography		43

List of Figures

2.1	Rainbow key structure [1] (F is the central map, S and T are used for hiding the special structure of F (together with which they form the private key) and P is the public key)	8
2.2	Rainbow sign algorithm [1]	10
3.1	Schema of GHSMITH [2]	13
3.2	GHSMITH basic cell [2]	14
3.3	GHSMITH pivot cell [2]	15
3.4	GHSMITH pivot column cell [2]	16
3.5	GHSMITH pivot row cell [2]	16
3.6	Parameters for the HW implementations from 2008 and 2011 [3] .	19
4.1	Block diagram	22

List of Tables

2.1	Security levels and key sizes of Rainbow [4]	9
2.2	Parameters for different variants of Rainbow [5]	9
5.1	Key address map	31
5.2	Variable explanation	32
5.3	Temporary address map	33
5.4	Inputs and outputs of the modules	34
6.1	Used area according to the bus width	37
6.2	Number of needed clock cycles according to the bus width	38
6.3	Maximum frequency according to the bus width	38
6.4	Power consumption according to the bus width	39

Introduction

The number of devices connected to the internet has been increasing rapidly in the past years, and these devices need a secure way to communicate. At this moment, most of them are using either symmetric encryption algorithms or asymmetric public key algorithms such as RSA that rely on the difficulty of factoring the product of two large prime numbers for security. These public-key cryptosystems are considered secure under today's conditions. However, it is only a matter of time until a sufficiently large quantum computer is built. Then, the security of these cryptosystems will be significantly reduced as Shor's algorithm [6] can be used to factor numbers efficiently.

While a consumer-grade quantum computer may mean a breakthrough in disciplines such as medicine, it might be a disaster when it comes to cryptography. When a sufficiently large quantum computer is manufactured, it will render most of the RSA-based schemes unsafe. This includes both integer factorisation-problem based ciphers as well as discrete logarithm-based ciphers. Even though we do not possess a quantum computer that would be able to break currently used encryption yet, we need to prepare for the fact that such computers will become available sometime in the future.

For this reason, the National Institute of Standards and Technology (U.S. Department of Commerce) is running the Post-Quantum Cryptography Standardization Process. In this process, candidate encryption algorithms and signature schemes candidates are evaluated in several rounds.

When we take quantum computing into account, the most vulnerable ciphers use public and private keys. For example, AES is less vulnerable to quantum computing if a key of sufficient length is used – $2^{99.5}$ operations are needed for recovering a 256-bit key using Alex Biryukov and Dmitry Khovratovich's approach [7].

The Rainbow [8] scheme is based on the balanced and unbalanced Oil and Vinegar schemes; in fact, it is a generalisation of the Oil-Vinegar construction [8]. The original Oil-Vinegar scheme intends to provide systems with a small amount of RAM and processing power with a public key authentication.

Both the Oil-Vinegar and Rainbow schemes are based on an NP-hard problem of solving m quadratic equations for n variables. When m and n are almost equal (but not equal), this problem becomes very difficult, even for a quantum computer [9]. This means that Rainbow classifies as a post-quantum signature.

1.1 Goals

This work aims to implement Rainbow on FPGA, with the result being an Intellectual Property (IP) block that can be connected over the Advanced eXtensible Interface (AXI bus) and can serve as an accelerator in hardware/software codesign in a System on Chip. This thesis will be one of the first published hardware implementations of Rainbow, and, to our best knowledge, the implementation in the form of an accelerator is going to be unique.

The work will also serve as a base for future research regarding its side-channel leakage analysis.

This is the simple outline of this thesis. First, the reference implementation will be transferred to a development kit containing both an ARM core as well as FPGA. Next, an AXI connected accelerator will be developed in a Hardware description language (HDL) based on the reference implementation. Lastly, the performance improvements and usage of resources will be evaluated.

Theoretical background

Cryptography provides us with four basic services which are widely used today. These are

- confidentiality (no one can eavesdrop),
- authentication (way of identification),
- data integrity (data are delivered without any errors), and
- non-repudiation (no one can take back what was sent).

Let us say I want to send a picture of my cat to a friend, and I do not want anyone else to see it. We both generate our RSA keypairs, and we publish the public keys. I can then use the public key of my friend to encrypt the image and send it to my friend, who would use the private key to decrypt it.

Alternatively, we can use RSA to exchange a key for a symmetric cipher such as AES. This way, there is one extra step compared to using just RSA, but the encryption/decryption is much faster when using a symmetric cipher. Or, we could use the Diffie-Helman method to agree on a key. In either case, we managed to transmit the message successfully, and nobody else could have seen the original content. This is the confidentiality aspect of cryptography. If instead of a cat picture, I sent a secret message previously known to my friend (such as a hashed password), I would have presented the authentication side as well.

If I want to make sure that my friend indeed got the picture of my cat and not a photo of my neighbour's one, I can calculate the hash of the image and publish it somewhere trustworthy. Then I can encrypt the message and send it to my friend. They then compare the hash of the decrypted message to the one I sent them. If it matches, they got the correct document, and we showed the data integrity aspect of cryptography. I could as well sign the image with my digital signature, which my friend can then verify. Then I can

prove that I was the one who sent the image and that I indeed sent it. This is non-repudiation (and authenticity).

2.1 Quantum computing

Computers today encode information in bits (zeros and ones) which, compared to quantum computers, restricts their abilities. Quantum computing uses quantum bits - qubits. These bits can exist in more than one state at the same time. Unlike classical computers, the power of quantum computer increases exponentially with each added qubit [10].

While classical computers are ideal for everyday tasks, quantum computers are ideal for running specific algorithms. With the correct algorithm, quantum computers can accelerate specific tasks extremely. However, this only applies to these specific tasks and specific algorithms [10].

2.2 Need for post-quantum cryptography

Security of all the public key cryptography discussed in the previous paragraphs is based on the assumption that the underlying function is one-way. This means it is easy to compute but no known algorithm is known to reverse it. Examples are the discrete logarithm or integer factorization with large primes. Here, mainly the discrete logarithm will be discussed. All this means that it is hard to find discrete logarithms on today's computers, but may be significantly easier using quantum computers.

In 2020, IBM released a quantum computer with 65 qubits, and they plan to make computers with 127 and 1121 qubits available in 2021 and 2023, respectively [11]. Given that NIST recommends at least RSA-2048 (2048-bit key) and the memory complexity of Shor's algorithm (see subsection 2.2.1), it is still not enough to crack today's encryption. However, if IBM manages to keep up with their schedule, they might be getting close to having a sufficient amount of memory in the foreseeable future).

As stated in chapter 1, it is mainly the asymmetric cryptography (which includes signature schemes) that's endangered. Symmetric ciphers such as AES were not proven insecure yet [7].

2.2.1 Shor's algorithm

Shor's algorithm is a quantum algorithm for efficient factoring of numbers. The algorithm allows a quantum computer to factor numbers a lot faster than if we would use conventional methods. In fact, the time complexity is just $O((\log N)^3)$ where N is the number of bits [6].

This is a very simple overview of how Shor's algorithm works [12]

1. Let N be the number we are trying to find factors of

2. Pick a random number g
3. $g^p = m * N + 1$
4. $(g^{p/2} + 1) * (g^{p/2} - 1) = m * N$
5. $(g^{p/2} + 1)$ and $(g^{p/2} - 1)$ are multiples of factors of N
6. Using Euclid's algorithm, these multiples of factors can be transformed into the real factors of N

If $p/2$ is not a whole number or if $(g^{p/2} + 1)$ or $(g^{p/2} - 1)$ is a multiple of N , the algorithm starts over [12]. For finding p , quantum principles, which are outside of the scope of this thesis. For example [12] explains these principles on the basic level.

Given the quantum computers abilities, it is clear that quantum computers pose a big threat to computer security. The only reason it is not possible to use the aforementioned quantum computers to break the encryption right now is the lack of memory (expressed in qubits). For factoring a number, Shor's algorithm needs approximately $O(3N)$ qubits where N is the number of bits the factored number has [13].

2.3 NIST Post-Quantum Cryptography Standardization Process

In 2016, NIST launched the Post-Quantum Cryptography Standardization Process. The deadline for submissions was in November 2017. There were 69 valid submissions that were reduced to seven during the first two rounds. The process aims to standardise a set of key encapsulation mechanisms and signature algorithms. Anyone can comment on the submissions, and NIST evaluates both security and performance [14].

There are several security levels the candidates can meet. For block cipher¹ replacements, the candidates need to have a security equivalent to a block cipher with a 128, 192 or 256-bit key when using a quantum computer for categories *I*, *III* and *V* respectively. When it comes to signature scheme replacements, these must have security equivalent to a 256 or 384-bit hash functions for categories *II* and *IV* respectively [15]:

NIST aims to standardise both public key and digital signature algorithm.

The public key (key-establishment) algorithm must conform to the IND-CCA2 property. That means the scheme *enables 'semantically secure'² encryption or key encapsulation with respect to adaptive chosen ciphertext attack* [15].

¹Ciphers that encrypt blocks of data of fixed length using a fixed key.

²Let P_1 and P_2 be plaintexts and C_1 and C_2 their respective ciphertexts. If an attacker cannot guess with higher than 50% probability which ciphertext corresponds to which plaintext, the scheme is said to be semantically secure [16].

The digital signature algorithm must conform to the EUF-CMA property [15]. This basically means that different messages will always produce different signatures

2.4 Algorithm types

The Standardisation Process candidates exploit several mathematical principles. This section presents summaries of these principles.

The first algorithm family is based on problems tied to lattices which are $n - \text{dimensional}$ periodic structures. All the lattice-based third round submissions rely on either the NTRUN-Th Degree Truncated Polynomial Ring (NTRU) problem or the Learning With Errors (LWE) problem. These are well described in [17]. Five of the third round candidates are lattice-based. All of them provide both signing and encryption while being relatively fast [14].

Another category consists of Error-correcting codes. As the name suggests, error-correcting codes (ECCs) were designed to provide less error-prone communication over a communication channel. Thanks to those codes, corrupted messages can be corrected without the need of sending them again. ECC based cryptography adds such errors on purpose and in a way that only those in possession of the private key can remove them. The main disadvantage of these schemes is the large public keys. Even the smallest variants are still larger than the lattice-based candidates. One candidate in the third round uses ECCs [14].

The third family are Multivariate Quadratic schemes which are based on solving sets of quadratic equations. Since solving such set is an NP-hard problem even for a quantum computer, only a person with the private key can find the solution. This is thanks to the special structure of these equations. They look random to an attacker, but they can be solved if the structure is known. These schemes produce very small signatures and can provide fast signing. The biggest disadvantage is the large (public and private) key size. Rainbow is the only candidate in the third round to use a multivariate quadratic scheme [14].

Hash-based signatures (HBS) are the next algorithm family. They consist of a collection of many one-time signature schemes and use a data structure to combine them effectively. Each of them is only used once [14].

The next family are elliptic curves which are widely used in asymmetric cryptography today; however, they are known not to be quantum-resistant. However, isogeny-based schemes use different arithmetics, so they are safe even from a quantum computer. The arithmetics are based on creating isogeny between two curves. There is one alternate candidate in the third round that uses this scheme. NIST believes that these schemes could benefit from further research [14].

Finally, some of the submissions did not clearly fall into one of the mentioned categories; however, just one of these was successful enough to be selected as an alternate candidate in the third round. However, the cryptographic community is not fully confident in its security [14].

2.5 Rainbow

The Rainbow scheme is based on the Oil-Vinegar system, which means that it relies on the aforementioned fact that solving m quadratic equations for n variables gets very difficult. [9] However, to solve these, the Unbalanced Oil-Vinegar (and Rainbow for that matter) scheme uses a special structure of polynomials.

2.5.1 Basic idea

The very basic idea of the multivariate signature schemes is to have a system of quadratic equations. Systems of quadratic equations are believed to be NP-hard. However, in this case, the system has a special structure so that we are able to solve it while it still seems quadratic to the attacker. When we solve this system, we get the signature which is easy to verify (because verifying a solution of a system of equations can be done in constant time) [18].

The Unbalanced Oil-Vinegar scheme consists of hiding quadratic equations in n unknowns called oil and v unknowns called vinegar over a finite field K , with affine secret functions [18]. For Balanced Oil-Vinegar schemes $n = v$ and for Unbalanced $v > n$. Next, we will consider only the unbalanced variant.

The basic variant of Oil and Vinegar uses a quadratic map F (finding F^{-1} is easy thanks to the special structure) and a linear invertible maps T and S , which is used to hide the structure of F . The public key P is the composition of these three maps. The map F is also called the Central map. All these maps are constructed in a finite field [19].

To get a signature $x = P^{-1}(y)$ we need to first compute $z = S^{-1}(y)$, then $w = F^{-1}(z)$ and then $x = T^{-1}(w)$. To do this, we randomly generate the v vinegar unknowns, and then we compute the oil unknowns from S [18]. Next, we solve the remaining linear equations using Gaussian elimination. If a solution exists, we are done. If it does not, we start over. Thanks to the special structure of F (no *oil * oil* terms in the polynomials), it is guaranteed that we will get a set of linear equations.

When we get the public key P , it seems that it is a random quadratic system since the Oil-Vinegar structure is hidden by T . There should always be more vinegar variables than the oil ones, but not by a significant margin. Ratios of about $v = 2o$ or $v = 3o$ are considered ideal [19].

Rainbow takes this principle and applies it in multiple layers. The scheme works basically the same as the Unbalanced Oil-Vinegar scheme, except that it

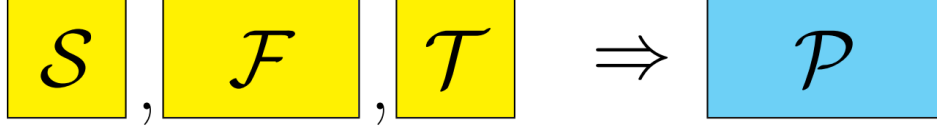


Figure 2.1: Rainbow key structure [1] (F is the central map, S and T are used for hiding the special structure of F (together with which they form the private key) and P is the public key)

adds more layers. A layer is a set of equations and variables, that is processed individually. When additional layers are added, security increases.

Also, the Rainbow signature is much smaller than the previously described Oil-Vinegar variants. The signature is only slightly longer than the digest of the document [8].

These schemes are efficient when it comes to RAM and required processing power but quite inefficient when the public key size is considered. However, private key computations may be performed without the public key [18].

The parameters (number of equations, number of variables in each layer and the number of layers) will be discussed later.

2.5.2 Key structure for Rainbow

According to [1], private key consists of

- two invertible affine maps $S : F^m \rightarrow F^m$ and $T : F^n \rightarrow F^n$
- the quadratic central map $F : F^m \rightarrow F^n$, consisting of m multivariate polynomials $f^{(v_1+1)}, \dots, f^{(n)}$.

According to [1], the public key is the composed map

$$P = S \circ F \circ T : F^n \rightarrow F^n$$

The structure of the keys can be seen on Figure 2.1.

2.5.3 Security levels

The classic variant³ of Rainbow implements three different security levels. Table 2.1 presents those security levels along with key and signature sizes.

³There is also a cyclic variant which is out of the scope of this thesis. Its main advantage is the reduced public key size.

Level	Public key size (kB)	Private key size (kB)	Signature size (bit)
I	151.8	101.2	528
III	861.4	611.3	1312
V	1885.4	1375.7	1632

Table 2.1: Security levels and key sizes of Rainbow [4]

2.5.4 Parameters for Rainbow

The Rainbow signature scheme competes in three security categories (discussed in subsection 2.5.3 and section 2.3). For each of these categories, Rainbow uses different parameters [5].

The first parameter is the size of the Gallois Field. Category 1 uses $GF(16)$ and categories 3 and 5 use $GF(256)$. Next are the numbers of vinegars and oils for each layer. These parameters not only affect security but also the key size [5]. The parameters can be seen in Table 2.2 and the security levels are in Table 2.1.

Level	Parameters (F, v_1, o_1, o_2)
I	$GF(16), 36, 32, 32$
III	$GF(256), 68, 36, 36$
V	$GF(256), 92, 48, 48$

Table 2.2: Parameters for different variants of Rainbow [5]

2.5.5 Algorithm as presented by its authors

The algorithm for signing a digest can be seen on 2.2. The algorithm was taken from [1] and was not modified in any way. Rows 1 till 5 describe pre-computation for the first layer. Rows 6 to 14 show the main signature process and the remaining rows show the final processing of the output.

2. THEORETICAL BACKGROUND

Input: document d , Rainbow private key $(InvS, c_S, \mathcal{F}, InvT, c_T)$, length ℓ of the salt

Output: signature $\sigma = (\mathbf{z}, r) \in \mathbb{F}^n \times \{0, 1\}^\ell$ such that $\mathcal{P}(\mathbf{z}) = \mathcal{H}(\mathcal{H}(d)||r)$

```

1: repeat
2:    $y_1, \dots, y_{v_1} \leftarrow_R \mathbb{F}$ 
3:    $\hat{f}^{(v_1+1)}, \dots, \hat{f}^{(n)} \leftarrow f^{(v_1+1)}(y_1, \dots, y_{v_1}), \dots, f^{(n)}(y_1, \dots, y_{v_1})$ 
4:    $(\hat{F}, c_F) \leftarrow \mathbf{Aff}^{-1}(\hat{f}^{(v_1+1)}, \dots, \hat{f}^{(n)})$ 
5: until  $\text{IsInvertible}(\hat{F}) == \mathbf{TRUE}$ 
6:  $InvF = \hat{F}^{-1}$ 
7: repeat
8:    $r \leftarrow \{0, 1\}^\ell$ 
9:    $\mathbf{h} \leftarrow \mathcal{H}(\mathcal{H}(d)||r)$ 
10:   $\mathbf{x} \leftarrow InvS \cdot (\mathbf{h} - c_S)$ 
11:   $(y_{v_1+1}, \dots, y_{v_2}) \leftarrow InvF \cdot ((x_{v_1+1}, \dots, x_{v_2}) - c_F)$ 
12:   $\hat{f}^{(v_2+1)}, \dots, \hat{f}^{(n)} \leftarrow \hat{f}^{(v_2+1)}(y_{v_1+1}, \dots, y_{v_2}), \dots, \hat{f}^{(n)}(y_{v_1+1}, \dots, y_{v_2})$ 
13:   $t, (y_{v_2+1}, \dots, y_n) \leftarrow \text{Gauss}(\hat{f}^{(v_2+1)} = x_{v_2+1}, \dots, \hat{f}^{(n)} = x_n)$ 
14: until  $t == \mathbf{TRUE}$ 
15:  $\mathbf{z} = InvT \cdot (\mathbf{y} - c_T)$ 
16:  $\sigma \leftarrow (\mathbf{z}, r)$ 
17: return  $\sigma$ 

```

Figure 2.2: Rainbow sign algorithm [1]

System overview

This chapter presents an analysis of the task. First, the algorithm is analyzed. After that, the design is broken into components and communication of the components is discussed. Lastly, existing hardware implementations of Rainbow re presented.

3.1 Algorithm description

This section contains an analysis of the reference implementation. Anything related to random number generation and hashing is omitted as these operations are out of the scope of this thesis, which focuses solely on the Rainbow algorithm (therefore lines 2, and 9 in Figure 2.2 are omitted).

First, the randomly generated vinegar variables of the first layer are populated into the central map. This is done using matrix-vector multiplication which also includes the multiplication of a vector by one element. Then, the part of the central map that belongs into the first layer with regards to these vinegars is inverted. If the inversion is successful, the algorithm proceeds to the next stage. These operations are described by lines 1 – 6 in Figure 2.2.

After that, the remaining operations for the first layer are executed. This includes multiplication by the S matrix and solving the equations for the first layer. These operations are described by line 10 in Figure 2.2.

Next, the variables needed for the second layer are computed. In this phase, the module for matrix-vector multiplication is reused, but a module for evaluating triangular matrices is also needed. These sets of equation will look quadratic to the attacker, but thanks to the special structure, they can be easily solved. These operations are described by line 11 in Figure 2.2.

The third block of operations contains the main operations over the central map. The operations needed are matrix-vector multiplication, vector addition, copying memory, resetting memory and triangular matrix evaluation. Last but not least, a set of 32 equations (with 32 unknowns) must be evaluated. If

the evaluation is successful, the algorithm proceeds to the last stage. These operations are described by lines 12 – 13 in Figure 2.2.

In the last stage, operations over the T map are performed. Only matrix-vector multiplication, vector addition and memory copy modules are needed. These operations are described by lines 15 – 17 in Figure 2.2.

3.2 Design

From the above analysis, the conclusion was made, that these modules are needed.

- Module for multiplying a vector by one number
- Module for merging the layer one vinegars into the central map
- Module for matrix-vector multiplication
- Module for evaluating the triangular matrices
- Module for copying memory
- Module for resetting memory
- Linear equation solver
- Main controller

All of these modules are needed on the PL⁴ side. The linear equation solver seems like the most complex module. Besides these modules, controller application is needed on the ARM side. This application should handle the communication between the ARM and PL sides.

Again, this section avoids any modules that involve either hashing or generating random numbers. These can be done independently in the ARM CPU and are not part of this work.

3.2.1 Linear equation solver

A linear equation solver is needed two times during the signing process. One time we actually need to solve a set of linear equations, and the second time a matrix inversion is needed.

The linear equation solver deserves special attention. A regular implementation of the Gauss-Jordan elimination would be too slow for our purpose. One of the state of the art approaches is the GSMITH algorithm. It is highly parallel and allows for quick computation [2].

⁴Programmable logic

There are multiple variants proposed by the authors in the same paper but due to data handling and speed, the original GSMITH seems best for this purpose.

Solver schematic can be seen on Figure 3.1. The subsections below discuss the components of the solver, which can be seen on this schematic.

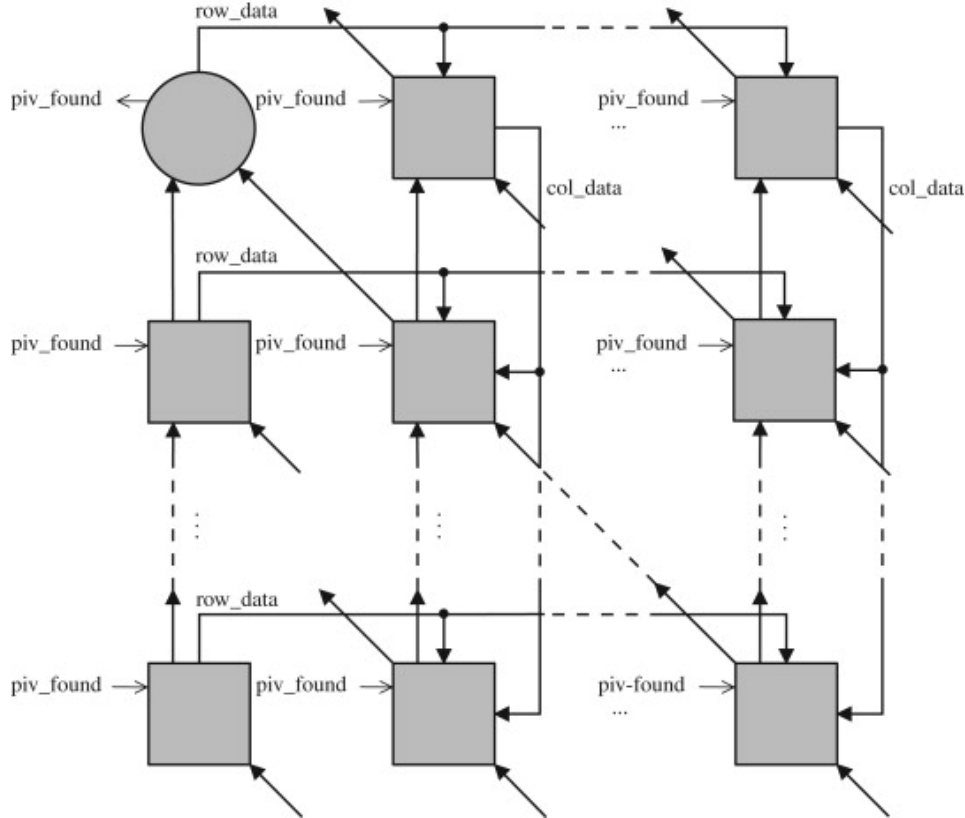


Figure 3.1: Schema of GHSMITH [2]

Before the main components of the solver are discussed, few words about registers should be said. When a basic register is mentioned, it means a register that can hold one $GF(16)$ element. The register has a load enable signal as well as a reset signal. There is also an another register, which is a simple one-bit register with load enable and reset signals.

3.2.1.1 Overview of the solver's design

If the cell in the top left is not zero (pivot is found), the row is normalised, and the eliminate step is executed. In this step, all the cells execute their own operations and send the result to their upper left neighbours. Therefore all the columns effectively shift left, and all the rows shift right. After this

3. SYSTEM OVERVIEW

happens, the last row does not need to be used anymore, and its `lock_row` flag is set to 1 [2].

If the pivot is zero, all the rows that are not locked are cycled in the upwards direction. The same operation can be used for loading the solver. Therefore, the solver itself does not need a controller if it can be reset before each operation. To load the solver, the data for the first equation are connected to the bottom-most row's input. Since the pivot (top-left cell) is now zero (after reset), the global signal *piv_found* which determines whether pivot has been found will also be zero. This signalizes that the top row needs to be pushed all the way down and the remaining rows should be pushed up. During loading, the first row is discarded. This allows for very easy loading which essentially reuses the operation for rotating the rows and looking for the new pivot.

3.2.1.2 Basic cell

The basic cell is the most complex and the basic stone of the solver at the same time. The cell has connections to four of its neighbours and some global connections [2]. The schematic can be seen in Figure 3.2.

The cell has connections to its upper and lower neighbours as well as its lower right and upper left neighbours. It is also connected to the global network that provides information on whether a pivot has been found, a signal which stores whether that row should be locked as well as connections to the row and column data buses [2].

The cell contains a basic register, a $GF(16)$ multiplier and a $GF(16)$ adder. The data going into the register is selected based on the global signals. It is implemented as one module that is instantiated many times.

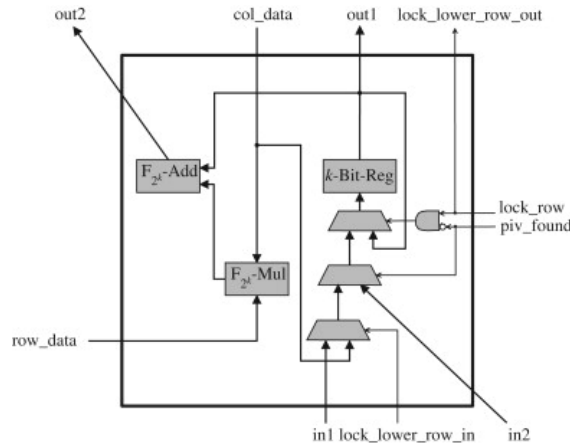


Figure 3.2: GHSMITH basic cell [2]

3.2.1.3 Pivot cell

The pivot cell is responsible for inverting the pivot. Therefore it is placed in the top left corner of the solver system. It contains only the $GF(16)$ inverter, and it outputs the inverted pivot on the row data bus as well as the global signal saying whether the pivot was found [2]. The schema is on Figure 3.3. The row data are used for the normalisation. The module is instantiated just once.

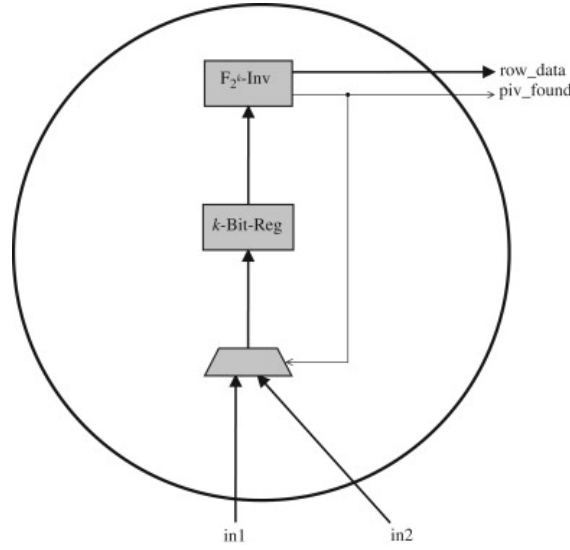


Figure 3.3: GHSMITH pivot cell [2]

3.2.1.4 Pivot column cell

The pivot column cells sit in the top row and provide the data on the column data bus. These data are used for elimination.

The cell contains just a basic register that is loaded based on the global signals. The schema can be seen on Figure 3.4. The cell is instantiated $2 * number_variables - 1$ times.

3.2.1.5 Pivot Row cell

The pivot column cells are placed in the column under the pivot cell, so the module is instantiated $number_variables - 1$ times. The cell contains one basic register and a $GF(16)$ multiplier. It provides data on the row data bus, which is used for elimination. The register is loaded based on the global signals, and the schema can be seen on Figure 3.5.

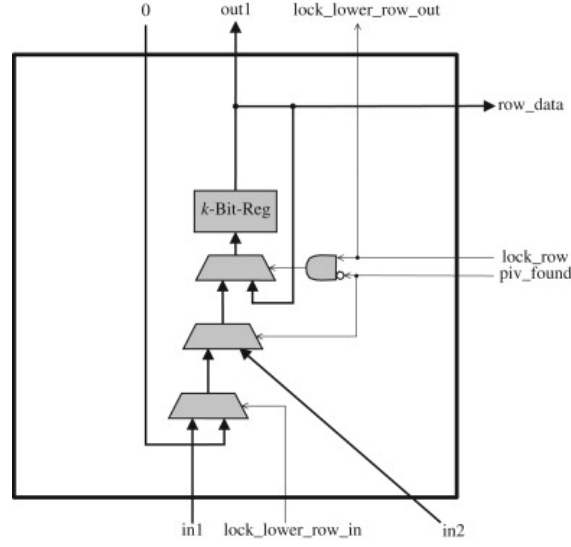


Figure 3.4: GHSMITH pivot column cell [2]

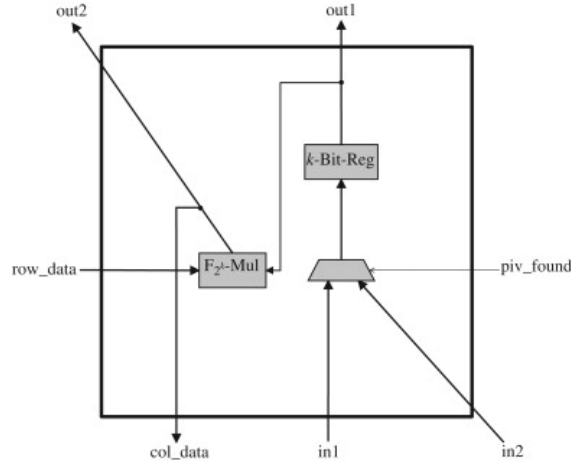


Figure 3.5: GHSMITH pivot row cell [2]

3.2.2 Modification of the GSMITH algorithm

This subsection discusses the modifications that were done to the linear equation solver (GSMITH).

3.2.2.1 Registers

There are two types of registers. The first one can hold either one or two $GF(16)$ elements. Under normal circumstances, just the first element is used. The second use case is discussed in subsubsection 3.2.2.3. There is a load

enable signal as well as a reset signal. One can choose which element is being loaded/outputted using another signal. Except for this small adjustment, the register behaves normally. In this whole subsection, this register will be called the basic register.

The second type of register is a simple one-bit register with load enable and reset signals.

3.2.2.2 Wrappers

There are two wrappers for the solver. The first one instantiates all the cells and provides connections between them.

All the connections mentioned in subsection 3.2.1.1 are handled in the first wrapper, as well as a counter that is reset every time the normalise operation takes place. This is to prevent an everlasting loop if a set of equations is not uniquely solvable.

The aforementioned wrapper needs an entire row of data to load it. In the case we have 32 variables ($GF(16)$ elements), we need to load either 128 or 132 bits of data depending on the value of the right side (if it is zero for the entire set, there is no need to load it). Therefore the outer wrapper implements loading of such data, and it also controls the write enable flags for the registers, so the data are loaded at the correct time.

The outer wrapper also implements the reading of the result and the write-back to the memory. This is also the first entity that implements an FSM. The entire solver and the first wrapper have no need for it. It also detects if the computation was successful and exposes this information to a higher level entity.

3.2.2.3 Modification for inversion

At one point in the computation, a matrix inverse is needed. The easiest way to calculate the inverse of a matrix larger than 5x5 is using the formula

$$\begin{pmatrix} I & | & M \end{pmatrix} \sim \begin{pmatrix} N & | & I \end{pmatrix}$$

where M is the original matrix, N is its inverse, and I is the identity matrix. This exact same process was implemented, and it is where the basic registers from subsection 3.2.2.1 come to play.

Instead of doubling the whole solver, only the registers were doubled. Therefore if the module is set to inversion mode, the first operation takes place as usual. However, the right outputs are stored in registers which are connected to the input of the second column of cells (first column of basic cells) in the next clock cycle. The FSM also sets the global signals for the registers, which determines which element should be loaded/outputted. In the end, if the matrix is invertible, the result is stored in the body of the original solver (identity matrix gets shifted out). The result is then sequentially shifted out

and stored in the memory at the specified address. All this is handled in the outer wrapper.

3.3 Communication

The industry standard for connecting the ARM processor and the FPGA sides is the use of the AXI bus. There are three types of the AXI bus - AXI, AXI lite and AXI streaming. The AXI stream interface has no addressing and allows unlimited data transactions between the slave and the master.

AXI is a multi-master, multi-slave synchronous parallel bus, designed for on-chip operations.

Both AXI and AXI lite use memory mapping. The main difference is that AXI lite allows for just single bit memory map transactions. This has very low overhead and is ideal for control signals. The full-fledged AXI allows memory map burst transactions [20].

3.4 Related research

There are three published papers that focus on hardware implementations of Rainbow to the author's best knowledge. The first paper was published in 2008, and its authors designed an architecture that can sign a digest in 804 clock cycles [3]. It was followed by [21] in 2011, where the signature process is four times faster.

However, these implementations use different parameters to the version of Rainbow that was submitted to the NIST standardisation process. In these cases, the digest and the signatures are only 24 and 42 bytes long, respectively.

The last paper was published in 2018, and despite the architecture being slower (1980 clock cycles for the Ia classic variant), it brings more value to the table since it can work over different finite fields as well as with different sets of parameters [22].

3.4.1 Approach of S. Balasubramanian et al.

This team of authors developed an architecture that consists of approximately 64 thousand gate equivalents and signs a digest in 804 clock cycles at 67 MHz in a Xilinx Virtex 4 FPGA. The authors use GSMITH ([2]) as the linear equation solver, but they also adapted it to perform matrix-vector multiplication and vector addition. They use column-wise pipelining in the solver (shifting already used parts of the matrix out from the leftmost column and shifting new matrix data into the rightmost one), which essentially masks all memory access induced latency [3]. The parameters for this implementation can be seen on Figure 3.6

Parameter	Rainbow
Ground field size	$GF(2^8)$
Message size	24 bytes
Signature size	42 bytes
Number of layers	2
Set of variables in each layer	(17, 12), (1, 12)

Figure 3.6: Parameters for the HW implementations from 2008 and 2011 [3]

3.4.2 Approach of H. Yi and Z. Nie

Differently from the previous paper, authors of this architecture developed their own linear equation solver. Besides that, they developed modules for polynomial evaluation and affine transformations, which handle all other operations. Their inverter is based on the Fermat theorem, and they developed a new type of multiplier that multiplies three elements instead of two. All this allowed the authors to create a highly efficient and highly parallel structure [21]. The parameters (which are the same as in the previous case) for this implementation can be seen on Figure 3.6. This implementation is roughly four times faster at 198 clock cycles.

3.4.3 Approach of A. Ferozpur and K. Gaj

This approach is the only one that can be used for different sets of parameters and different fields. When run with the same parameters as [3] and [21], it produces a signature in only 148 clock cycles. With parameters for the version Ia classic, the signature generation takes 1980 clock cycles. The architecture uses a scheduler that provides the correct signals to their own linear equation solver along with the modules for affine transformations and polynomial evaluations. The authors mention that they focused on minimising the latency as much as possible [22].

Accelerator design

This chapter discusses the basic design of the accelerator.

4.1 Block diagram

The design was split into several modules. The first one is called *gfm_{at}_prod* and handles both the matrix-vector multiplication as well as the merging of the vinegars into the central map.

Next, there is the module called *batch_quad_trimat_eval* which takes care of evaluating the triangular matrices.

After that, vector addition is needed. It is called *gf256_add* (a single $GF(256)$ can add two $GF(16)$ elements at the same time without any modification, thus the name). This module needs to add two words that are *BUS_WIDTH* long (so they contain several elements). In the case of $GF(16)$ and $GF(256)$, addition means just bitwise *xor*.

Then there are modules for copying memory and resetting it. The *memcpy* module copies data from one memory address to a second one while the *reset_mem* module just writes zeros over all the bits in the specified range.

The most complicated module is, by far, the linear equation solver. Its structure is described in subsection 3.2.1.

Last but not least, the controller that is responsible for issuing all the control signals is needed.

Most of the modules are named after the functions from the reference implementation that perform the same job. All modules handle the loading of the needed data from the block RAM by themselves.

The block diagram of the design can be seen on Figure 4.1.

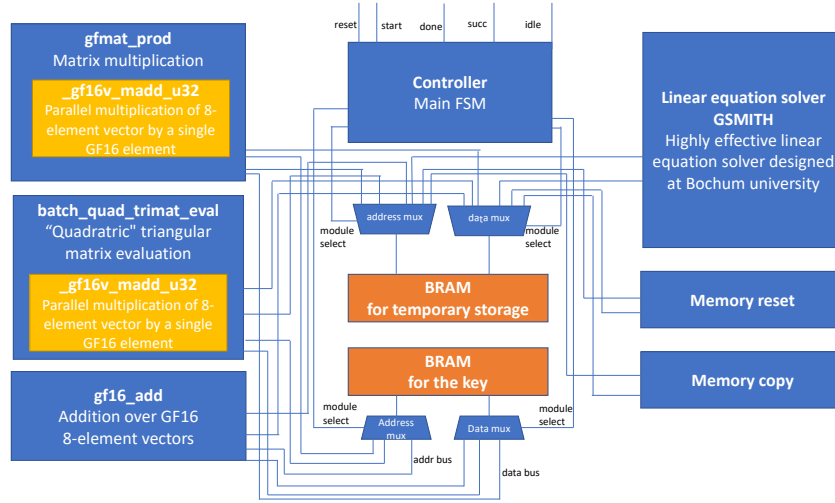


Figure 4.1: Block diagram

4.2 Parallelisation

Parallelisation is the key to speedup. Therefore the design of the accelerator takes advantage of as many parallel structures as possible.

The part that benefits from parallelisation the most is the multiplication. The main multiplier implements $\frac{BUS_WIDTH}{ELEMENT_SIZE}$ individual $GF(16)$ multipliers. Where BUS_WIDTH can be specified pre-synthesis and directly affects the number of LUTs used. Element size is 4 (for $FG(16)$). Therefore every time a chunk of data is loaded from the memory, all of it can be multiplied in the next clock cycle.

Adding is also done in the same way as multiplication – all loaded bits are added in one clock cycle.

Next up is the linear equation solver. When GSMITH is used, it is a highly parallel structure by itself. Therefore there is no need for extra parallelisation.

The modules load the data they need from the BRAM, process them and write back the results. When data from both the key ROM and the all-purpose RAM are needed in one step, the data are loaded at the same time, saving on clock cycles.

Latency masking is another way of speeding the design up by loading data from the memory while performing operations on other, already loaded, data. Every time some data must be loaded and processed, latency masking is implemented. For example, the module for matrix-vector multiplication first loads the first chunk of data, and when it starts processing it, it also starts loading the next chunk. Therefore, the latency of memory operations is only visible during the first loading cycle. Same goes for *batch_quad_trimat_eval*

and addition. Latency masking is used only in the modules themselves and not between them.

4.3 Communication

When communicating with the accelerator, three types of transactions need to be made.

At first, data must be sent to the accelerator. These data are the rolled vinagars and the salted hash. Then, if the computation is successful, the signature must be read. Both of these transactions can be easily implemented using the AXI bus. Thanks to library functions provided by Xilinx, it is very easy to write data directly from the ARM CPU over AXI to the block RAM.

For sending the control signals, AXI lite is used as it is the better choice as it has a lower overhead.

4.4 Chosen variant

There are three variants of the classic version of Rainbow. Each one has different parameters and security margins. These are discussed in subsection 2.5.4. The Ia variant was chosen as a basis variant for this accelerator as it requires the least amount of resources (field elements are smaller). However, if the parameters in the configuration package are changed, it is possible to use this design for any variant with any parameters as long as the used FPGA is large enough.

4.5 Parametrisation

The whole accelerator is designed to be parameterisable. One can select different bus widths, number of variables for each type and layer or the number of multipliers that run in parallel. These tightly follow the reference implementation, and anything that can be changed there can also be changed in this design. This includes the number of vinegars and oils, and with a bit of work, the Gallois field can be changed as well (the register widths need to be changed in the configuration package, and the multiplier needs to be replaced for this). It is also possible to change the memory or key structure completely. These settings are defined in the configuration and address package. For changing the memory interfaces, one also needs to edit the entity parameters.

There are few constraints:

- The maximum bus width is 64.
- The number of concurrent multipliers must be

$$\frac{BUS_WIDTH}{GF_ELEMENT_SIZE}$$

- Each part of the key must begin at an address that is divisible by 64.

The reason for parametrisation is the planned evaluation. Thanks to this, it will be possible to measure used area, processing time, maximum frequency and power consumption for different sets of parameters and compare it to the reference implementation running on the ARM side.

4.6 Linear equation solver

Since a basic implementation of Gaussian elimination would be slow (and a purpose of a dedicated accelerator is to speed up the signing process), we decided to use GSMITH, which is a linear equation solver that is fast when using not so large amount of variables. Its version for Gallois Field computation offers an even greater speedup [2]. However, this comes at the cost of the FPGA area. The whole modified solver, as discussed in subsubsection 3.2.2.3 uses about 50% of the available LUTs⁵.

4.7 Separation to modules

The design was separated into seven different modules (plus two IPs for BRAM). These modules mostly copy the reference implementation.

The first two copy the functions for matrix-vector multiplication (`gfmtat_prod`) and for evaluating a triangular matrix (`batch_quad_trimat_eval`). Those two modules also instantiate a submodule for multiplication in $GF(16)$ as discussed in section 4.2.

The third one is the module for adding numbers over $GF(16)$. The next modules implement copying the content of BRAM and resetting it (which is not possible to do in one clock cycle on this FPGA), respectively.

The most complex module is the linear equation solver, and the module set is completed with the main controller (FSM).

⁵Look-up tables

Implementation

This chapter discusses all the design choices and describes the method used.

5.1 Platform and language choice

For the design, we chose the ZedBoard⁶ which has the Zynq-7000 series SoC (XC7Z020-CLG484).

The board has 85 000 logical cells available, as well as 560KB block RAM. These parameters are ideal since the private key can be stored in the block RAM, and the number of logical cells should be sufficient. There is also an ARM Cortex A9 processor onboard, which allows easy performance comparison and the available AXI connection between the PL⁷ and PS⁸ sides is ideal for this codesign.

The accelerator was developed in VHDL. For the ARM core, the Reference implementation was used. It is written in C, and it was slightly modified to run on the ARM processor.

5.2 Top-level memory implementation

The design has to use either two or three memory types depending on whether the key is stored in ROM or needs to be dynamically changed and therefore stored in RAM.

The first memory type are registers that use the on-chip flip-flops.

Next is the BRAM, which is used as temporary memory. The individual modules load content from this temporary memory into their registers so they can work with it. The whole memory block needs to be reset to zeros before

⁶<https://www.avnet.com/wps/portal/us/products/avnet-boards/avnet-board-families/zedboard/zedboard-board-family>

⁷Programmable logic

⁸Processing system

the accelerator is run (this copies the reference implementation). Xilinx IP block was used for implementing the BRAM. The RAM is dual-port; therefore writing into it can happen at the same time as reading which is used for masking the memory latency.

The key can either be stored in BRAM (which allows the uses of different keys during one run of the program) or in ROM (which is easier to implement). In this case, ROM was selected, and the memory was also implemented using a Xilinx IP block. The key storage structure is the exact same as the reference implementation, with one exception. The key storage is modified that each part of the key always starts at an address that is divisible by 64.

5.3 RAM/ROM structure

This section discussed the structure of the key and temporary memory blocks.

5.3.1 Key memory

The memory block storing the key is 64 bits wide. The data are stored in *BUS_WIDTH* long blocks. This means that each block can be addressed and read within one memory cycle. If the *BUS_WIDTH* is changed, the number of blocks needs to be changed in the Xilinx IP configuration. However, everything else is recalculated automatically.

As mentioned in section 5.2, the memory is structured in such way, that blocks (and *BUS_WIDTH*s) of up to 64 bits in size can be used. Because the stored data do not always align into 64 bit wide blocks, padding with zeros is used. Since the key structure copies the reference structure ([5]) except for the aforementioned padding, blocks of the key can be transferred directly from the reference implementation into the RAM/ROM. However, due to the padding, transferring the whole key is not supported.

Table 5.1 shows the address map and Table 5.2 shows descriptions and values for the variables.

5.3.2 Temporary memory

For the temporary memory, exactly the same apply as for the Key memory. However, all these data are always stored in RAM.

There are three special blocks in this memory. First one is located at address zero and it is the designated space for the digest that is to be signed. Right after the digest, the generated vinegars for the first layer are stored. The last special block called *w* is the output block. When the accelerator is done, it stores the output in here. All these blocks can be seen in Table 5.3.

Table 5.3 shows the address map and Table 5.2 shows descriptions and values for the variables.

5.4 Implementation of modules

This section discusses the implementation of individual modules in detail. The ports of the modules can be seen in Table 5.4.

5.4.1 Registers

This section contains the details on all registers except for those that are used in the linear equation solver.

5.4.1.1 4096-bit register

The largest register needed is 4096 bit wide. It stores $4096/BUS_WIDTH$ data chunks that are BUS_WIDTH long, which allows easy loading from the memory. The register can either be loaded by the aforementioned chunks (address needs to be provided too) or as a whole. Each operation uses a different flag, and if both of the flags are on, loading of the whole register is prioritized. The register outputs the whole 4096-bit content on its output port.

The reason for storing the data in chunks instead of one large signal is the loading. If the whole register was addressed, there would be a need for a 4096-bit adder. This way, there can be just one adder with BUS_WIDTH bits and one adder with $4096/BUS_WIDTH$, which results in a very significant increase of maximum frequency and decrease in area. The whole accelerator design got about ten times smaller after switching from addressing of the whole signal to data chunks.

5.4.1.2 128-bit register

The 128-bit (16-byte) register is a copy of the 4096-bit one, except it stores just 128 bits of data. Therefore the old design was kept, and there are no data chunks.

5.4.1.3 144-bit register

The 144-bit (18-byte) register is a little different. It stores the data the same way as the 16-byte register, and it implements the same functions, but since the number of bits is not divisible by 32 or 64 (the planned largest bus widths), some modifications had to be made. If a value is loaded to the highest bits, the address is checked, and in case it is out of range, the data are ignored. For example, if the bus width is 32 bits, only the first half of the word is stored in the last load cycle.

5.4.2 GF16 Multiplication

This module takes two 4096-bit signals and one 4-bit signal as its inputs. The first large signal is the accumulator, and the second is a chunk of the matrix that is being multiplied. 4096 bits is the maximum size the accelerator may need to multiply in one go. This width is needed just few times during the computation but is necessary for the correct function. The last signal is the $GF(16)$ element that is being used for multiplication.

The multiplication itself is implemented as a lookup table. There is the correct number of individual multipliers to multiply all the $GF(16)$ elements in one *BUS_WIDTH* long word within one clock cycle. The multiplication result is added to the accumulator directly after multiplication.

The module makes use of the load signals of the registers and loads a chunk of the result in every clock cycle.

The module is controlled by its own FSM, which exposes ready and busy flags to the module above.

5.4.3 Matrix multiplication

First, this module loads the first chunk of the matrix that is being multiplied to its register. Then, it loads the vector of values that are used for the multiplication. Next, it begins the computation.

For the multiplication itself, the module from subsection 5.4.2 is used. While this module runs, another chunk of the multiplied matrix is loaded to another register. When the multiplication is done and the result is stored in the accumulator, the matrix registers are swapped, and the process begins anew. Thanks to this principle, all of the memory access latency, besides the initial loading and the last writeback, is masked. At the same time, thanks to the fact that the number of $GF(16)$ multipliers correspond to the *BUS_WIDTH*, the times for loading and multiplication differ just by two clock cycles.

When the process is done, the final result is stored in the accumulator register. This result is then written to the memory address specified in one of the module's inputs.

This module is controlled by an FSM that handles the handshake with the instantiated multiplier and provides flags to the top-level module. The entire module takes up approximately 10% of the available LUTs.

5.4.4 “Quadratic” triangular matrix evaluation

Similarly, as the matrix-vector multiplication module, it also loads data from the memory. Precisely, it loads chunks of the matrix and a vector for multiplying by (the same as in *gmat_prod*).

The point of this module is to evaluate the triangular matrix. This happens once for the first layer and twice for the second. The matrix has the

aforementioned (in section 2.5) special structure, so it is possible to evaluate it, but to an attacker, this would look like a set of quadratic equations. That is the reason the word quadratic is in quotes in the heading.

It does so in two nested loops controlled by counters and the FSM. When the outer loop is started, a temporary accumulator is reset. Then the inner loop runs from the current index of the outer loop up to the size of the multiply-by vector. In this inner loop, the multiplier from subsection 5.4.2 is used to iterate through the matrix. Again, latency for loading chunks is masked. When this inner loop is done, The same multiplier is used to store the contents of the temporary accumulator (multiplied by the corresponding value) into the actual accumulator. The outer loop runs through the whole multiply-by vector.

In the end, the contents of the accumulator are stored in the memory. Everything is, again, controlled by an FSM, which handles the handshake with the multiplier and controls which data are passed to it. Flags for higher-level entity are provided as well. The entire module takes up approximately 6% of the available LUTs.

5.4.5 Adder

The adder is a fairly simple module. It loads two *BUS_WIDTH* long words, adds them together and writes the result into the memory. Since it operates just over the BRAM used for storing the temporary results, the loading takes two clock cycles (one for each word). The adding and writeback take one more cycle. One of the loading cycle's latency is masked as it is executed during the writeback phase.

Adding in this particular field is very easy. All we have to do is *xor* the words.

The module is FSM controlled and provides status flags to the top-level module.

5.4.6 Memory reset

This is a very simple module. It sets the specified memory range to zero. Its state machine provides flags for the module above.

5.4.7 Memory copy

This module emulates the `memcpy` function from C. It simply copies data from one part of the memory to another. It can also copy data between different memory blocks. Flags for the top-level module are provided.

5.4.8 Controller

The main controller is one big state machine. It exposes control signals (both input and output) for the AXI-lite interface. After it receives the start signal, it goes through all the states, and if it detects an error, it falls to an error state. If the run is successful, it goes to a success state, and the data may be read. Both results need to be acknowledged before it goes back to the ready state.

5.4.9 GSMITH

GSMITH is the most complicated module. It consists of the solver itself and two wrappers.

5.5 Top-level connection and interface

All components were connected together with the controller in a top module (datapath.vhd), which exposes the FSM flags for AXI-lite communication plus the memory interface.

The memory was connected via AXI, which allows direct access from the PS side. For testing purposes, the salted digest is written directly in the memory from the C code, the start flag is set, and when the done flag is observed, the result is read directly from the memory. As a part of future work, this will be edited to use the AXI queue module that is available in the Vivado IP catalogue.

5.6 ARM implementation

The reference implementation was used directly on the ARM core for testing. Only a few modifications were made just to get the code to compile. The main problem was OpenSSL which just refused to compile in Xilinx Vitis. In the end, SHA-3 was used instead of the original hash function. This has no effect on the signature itself.

5.7 Input and output

As discussed in subsection 5.3.2, when the ARM chip wants to send the data to the accelerator, it does so via memory operations. It needs to write the generated vinegars and the digest to certain memory addresses. When that is done, it sends the start signal over the AXI lite bus.

Then, the ARM chip waits for the done flag. If the computation was successful, it can read the result from the memory. If the operation was not successful, new generation of vinegars is needed.

Name	Address	Description
sk_seed_ADDR	0	seed (not used)
s1_ADDR	sk_seed_ADDR + (LEN_SKSEED*8)/BUS_WIDTH	part of S map
t1_ADDR	s1_ADDR + (O1_BYTE*O2*8)/BUS_WIDTH	part of T map
t4_ADDR	t1_ADDR + (V1_BYTE*O1*8)/BUS_WIDTH	part of T map
t3_ADDR	t4_ADDR + (V1_BYTE*O2*8)/BUS_WIDTH	part of T map
i1_F1_ADDR	t3_ADDR + (O1_BYTE*O2*8)/BUS_WIDTH	part of C-map, F1, Layer1
i1_F2_ADDR	i1_F1_ADDR + (O1_BYTE*N_TRIANGLE_TERMS_V1*8)/BUS_WIDTH	part of C-map, F2, Layer1
i2_F1_ADDR	i1_F2_ADDR + (O1_BYTE*V1*O1*8)/BUS_WIDTH	part of C-map, F1, Layer2
i2_F2_ADDR	i2_F1_ADDR + (O2_BYTE*N_TRIANGLE_TERMS_V1*8)/BUS_WIDTH	part of C-map, F2, Layer2
i2_F3_ADDR	i2_F2_ADDR + (O2_BYTE*V1*O1*8)/BUS_WIDTH	part of C-map, F3, Layer2
i2_F5_ADDR	i2_F3_ADDR + (O2_BYTE*V1*O2*8)/BUS_WIDTH	part of C-map, F5, Layer2
i2_F6_ADDR	i2_F5_ADDR + (O2_BYTE*N_TRIANGLE_TERMS_O1*8)/BUS_WIDTH	part of C-map, F6, Layer2
KEY_END_ADDR	i2_F6_ADDR + (O2_BYTE*O1*O2*8)/BUS_WIDTH;	end address

Table 5.1: Key address map

Name	Value	Description
GFSIZE	16	Size of the GF
V1	36	Vinegars in L1
O1	32	Oils in L1
O2	32	Oils in L2
HASH_LEN	32	Size of the siged digest
V2	$((V1)+(O1))$	Vinegars in L2
PUB_N	$(V1+O1+O2)$	L1 vinegars + oils
PUB_M	$(O1+O2)$	Size of all oils
V1_BYTE	$(V1/2)$	Size of vinegars in L1
V2_BYTE	$(V2/2)$	Size of vinegars in L2
O1_BYTE	$(O1/2)$	Size of oils in L1
O2_BYTE	$(O2/2)$	Size of oils in L2
PUB_N_BYTE	$(PUB_N/2)$	Size of PUB_N
PUB_M_BYTE	$(PUB_M/2)$	Size of PUB_M
LEN_SKSEED	32	Seed size
SALT_BYTE	16	Salt size
SIGNATURE_BYTE	$(PUB_N_BYTE + SALT_BYTE)$	Signature size
MAX_O_BYTE	16	Maximum oil size
N_TRIANGLE_TERMS_V1	666	Size of the vinegar L1 matrix
N_TRIANGLE_TERMS_O1	528	Size of the oil L1 matrix

Table 5.2: Variable explanation

Name	Value	Description
vinegar_ADDR		generated vinegars
digest_salt_ADDR	0	digest
mat_l1_ADDR	$\text{vinegar_ADDR} + (((\text{V1_BYTE} + 6) * 8) / \text{BUS_WIDTH})$	matrix for L1
mat_l2_ADDR	$\text{digest_salt_ADDR} + (((\text{HASH_LEN} + \text{SALT_BYTE}) * 8) / \text{BUS_WIDTH})$	matrix for L2
r_l1_F1_ADDR	$\text{mat_l2_ADDR} + ((\text{O1} * \text{O2_BYTE} * 8) / \text{BUS_WIDTH})$	L1_F1 result
r_l2_F1_ADDR	$\text{mat_l2_ADDR} + ((\text{O2} * \text{O2_BYTE} * 8) / \text{BUS_WIDTH})$	L2_F1 result
mat_l2_F3_ADDR	$\text{r_l1_F1_ADDR} + ((\text{O1_BYTE} * 8) / \text{BUS_WIDTH})$	L2_F3 precomputation
mat_l2_F2_ADDR	$\text{r_l2_F1_ADDR} + ((\text{O2_BYTE} * 8) / \text{BUS_WIDTH})$	L2_F2 precomputation
z_ADDR	$\text{mat_l2_F3_ADDR} + ((\text{O2} * \text{O2_BYTE} * 8) / \text{BUS_WIDTH})$	
y_ADDR	$\text{mat_l2_F2_ADDR} + ((\text{O1} * \text{O2_BYTE} * 8) / \text{BUS_WIDTH})$	
x_o1_ADDR	$\text{z_ADDR} + ((\text{PUB_M_BYTE} * 8) / \text{BUS_WIDTH})$	
x_o2_ADDR	$\text{y_ADDR} + ((\text{PUB_N_BYTE} * 8) / \text{BUS_WIDTH})$	
temp_o_ADDR	$\text{x_o1_ADDR} + ((\text{O1_BYTE} * 8) / \text{BUS_WIDTH})$	
w_ADDR	$\text{x_o2_ADDR} + ((\text{O2_BYTE} * 8) / \text{BUS_WIDTH})$	output/accumulator
TMP_END_ADDR	$\text{temp_o_ADDR} + (((\text{MAX_O_BYTE} + 32) * 8) / \text{BUS_WIDTH})$	end address
	$\text{w_ADDR} + ((\text{PUB_N_BYTE} * 8) / \text{BUS_WIDTH})$	

Table 5.3: Temporary address map

Module	Direction	Width (bits)	Description
$GF(16)$ multiplication	in	4096	accumulator
$GF(16)$ multiplication	in	4096	multiplied data
$GF(16)$ multiplication	in	4	multiplier
$GF(16)$ multiplication	out	4096	output
Matrix multiplication	in	BUS_WIDTH	source address for multiplied data
Matrix multiplication	in	BUS_WIDTH	source address for multipliers
Matrix multiplication	in	N/A (integer)	length
Matrix multiplication	out	BUS_WIDTH	destination address for output
“Q.” triang. mat. eval.	in	BUS_WIDTH	source address for the data
“Q.” triang. mat. eval.	out	BUS_WIDTH	destination address for output
Adder	in	BUS_WIDTH	source address for data a
Adder	in	BUS_WIDTH	source address for data b
Adder	in	N/A (integer)	length
Adder	out	BUS_WIDTH	destination address for output

Table 5.4: Inputs and outputs of the modules

Testing and evaluation

This chapter discusses the testing and evaluation of the design.

6.1 Testing

All the modules and their submodules were individually tested using testbenches. The testbenches were written in VHDL and compared with the results from the reference implementation. For this purpose, the random data that would normally be generated during every run were generated just once. These fixed random data were then used for all the testing.

6.1.1 Reference implementation in ARM

First, the reference implementation was ported to run on the ARM core. This went without any major problems.

6.1.2 Memory access

The first challenge was to ensure coherent and fast memory access. The BRAM on chip has a latency of two clock cycles and large chunks of data are needed to perform the computations.

Since it is possible to simulate the IP block for BRAM in Xilinx Vivado, the included simulator was used to verify that the memory access was coherent. This turned out to be harder than expected. The original idea was to mask memory latency completely by loading the data for the next module while the previous one was writing its data to the memory. This idea had to be abandoned because it would require a large amount of work to synchronize everything correctly to prevent race conditions. As said in the previous chapter, latency masking is used only in the modules themselves and not between them.

6.1.3 Linear equation solver

The first implemented linear equation solver actually was not GSMITH but a systolic solver (without any particular name) by the same authors. This solver was easy to implement and did not require as many LUTs as GSMITH. The issue was that either the length of the critical path depended on the problem size (which was so large that the authors recommended against using the solver in this way) or required a special memory arrangement. In that case, the matrix would have had been loaded diagonally. This approach was tried and, due to the memory latency, was not used. Therefore, GSMITH was implemented.

GSMITH is the most complicated module from the entire accelerator. Both the implementation and the verification took large amounts of time. Especially connections of the edge cells are extremely error-prone as well as the deadlock detection.

6.1.3.1 Modification of GSMITH for inversion

The modification of the solver to perform both Gaussian elimination and matrix inversion took by far the longest time from the whole project. The idea behind the modification is very simple but the testing had to be extensive. Wolfram Mathematica was used to ensure correctness of the tests. Equations were generated using the reference implementation, converted using bash script to formats for both Vivado and Mathematica and results were compared, operation by operation. In the end, about 20 sets of equations were generated and tested. Besides these, edge cases (not uniquely solvable sets or equations with only zero elements) were tested.

6.1.4 Other submodules

Submodules other than the equation solver were fairly easy to test and the implementation was not problematic. Everything performs correctly and as expected.

6.1.5 Top-level module

The top-level module connects all the aforementioned modules together. During testing, some modules were synthesised but then they were cut out by the synthesis. Besides the usual errors in wiring, this turned out to be a problem with the equation solver. Because there was an error (some connections were flipped), the synthesis would cut out large parts of the main controller. This was very hard to debug as the error in the solver did not show during the usual testing on the per module basis. After these connections were fixed, everything synthesised correctly.

However, there is one error that has not been fixed yet. When everything is connected together and packaged as an AXI peripheral, the results are incorrect. Everything works without any issues before packaging the project as an AXI peripheral, though. All generated signatures verified as valid.

6.1.6 Different parameters

Changing the parameters - mainly the bus width - is possible from a configuration file. This works without any major issues (one constant must be defined directly in a file that is referenced from the package). The bus width directly effects the number of LUTs used and the computation speed.

6.2 Evaluation

Several different parameters were tested. These include the used FPGA area, power consumption, number of needed clock cycles and the maximum frequency. This chapter presents these testings in detail.

6.2.1 Testing in the ARM processor

First, the reference implementation was tested on the ARM core. One signature process took approximately 13.227 ms and 8809000 clock cycles at 666 MHz. This serves as the baseline for the next measurements.

6.2.2 Used area

The used area was measured in terms of number of LUTs⁹ and flip flops that together means how much of the FPGA resources had to be used. Table 6.1 presents these measurements. If the 32 bit version is taken as the baseline, it makes sense that the 16 bit version uses less resources. However, the amount of needed resources jumps up at 8 bits because a lot of LUTs are needed for address multiplexing and signal routing.

The differences in the number of used LUTs are not as significant as one might expect. This is due to the module for the linear equation solver having no components that can be affected by the bus width.

Bus width (bits)	Number of LUTs	Number of FF
8	40425	34962
16	35833	22533
32	39992	35190

Table 6.1: Used area according to the bus width

⁹Look-up tables

6.2.3 Number of clock cycles

Number of clock cycles was reported by the Vivado simulator. The less clock cycles, the faster the design. The clock frequency was set to the maximum possible frequency reported by the estimation tool in Vivado.

There is one issue, though. The reset of the temporary memory is taking a big portion of the needed time. This is due to my bad design and can be fixed by, for example, holding dirty bits for each memory block. The BRAM does not support resetting using any other method than writing zeros over the whole memory region.

Table 6.2 presents all measurements, with and without the time needed for resetting the memory. The number of clock cycles scales as expected (higher for lower bus widths).

Bus width (bits)	Clock cycles	Without reset
8	281000	119000
16	132000	67000
32	72000	39000

Table 6.2: Number of needed clock cycles according to the bus width

6.2.4 Maximum possible frequency and total computation time

Maximum possible frequency was reported by Vivado and the total time was calculated accordingly. Table 6.3 presents these measurements. The results look counterintuitive at first but after analysis, it was determined that the frequency can be higher at higher bus widths thanks to the fact that the synthesis tool does not need to insert so many addressing multiplexors.

Bus width (bits)	Maximum frequency	Total time / without reset (ms)
8	33.67 MHz	8.346 / 3.354
16	33.56 MHz	3.933 / 1.996
32	40.16 MHz	1.796 / 0.973

Table 6.3: Maximum frequency according to the bus width

6.2.5 Power consumption

Power consumption was reported by Vivado. Table 6.4 presents these measurements as well as the total power consumption per successful signature generation (without the reset). The power consumption corresponds to the amount of resources used. The total power used is reported in milliwatts per second.

Bus width (bits)	Power consumption (W)	Power used (mWs)
8	0.307	1.03
16	0.284	0.57
32	0.298	0.29

Table 6.4: Power consumption according to the bus width

6.2.6 Summary

Bus widths between 8 and 32 bits were tested. The number of clock cycles scales as expected and the more data can be transferred in one cycle, the faster the computation is. The memory reset needs redeveloping as it is taking a big amount of clock cycles.

The maximum frequency scales counterintuitively at first glance. However, as explained in subsection 6.2.4, this makes sense in the end.

The number of used LUTs scales as expected with the exception of the smallest bus width. There, more resources are needed. The power consumption scales in accordance to the amount of resources used.

In the end, it would have been better to test higher bus widths. Mainly at 64 and 128 bits. The accelerator was developed with up to 64 bit bus width in mind, however, this turned out to be non-functional due to insufficient parametrisation. In the future these issues will be fixed.

However, all working versions are faster than the baseline computation in the ARM CPU. If the reset time is not counted, the 32 bit variant is more than 13 times faster.

Conclusion

The goal of this thesis was the implementation of a parameterisable accelerator for the Rainbow signature scheme as well as testing how different parameters affect the synthesised design in terms of the area used, speed of computation and power consumption.

The goals were mostly achieved. The only thing that does not work is the AXI communication. Besides that, all modules work flawlessly. A valid signature is generated based on the digest and random numbers for the vinegars stored in BRAM. The data bus width can easily be adjusted from 8 to 32 bits in the configuration package.

The bus width is the main parameter to change. This directly affects the computation time and the consumed area (higher bus width means faster computation and more area used). If the bus width is set to 32 bits, the computation is more than 13 times faster than in the ARM CPU. Even if the smallest possible bus width is selected (8 bits), the computation is still faster than the ARM baseline.

The whole design fits into the used FPGA and the execution time scales linearly with the data bus width. The key structure was kept in accordance with the reference implementation.

Possible future modifications include modification of bus width to 64 and possibly even 128 bits. Also, a different approach to resetting the BRAM will be taken. This will allow an even bigger speedup. Besides this, since it is possible to swap the multiplier, the other variants of Rainbow can also be tested.

The main goal of developing a proof-of-concept parametrisable accelerator was achieved. The accelerator works as intended and when some small problems are solved, it can be used for future research.

Bibliography

- [1] Ding, J. *Rainbow*. Supporting documentation for the NIST contest submission. [cit. 2021-05-13]. Available from: <https://csrc.nist.gov/Projects/post-quantum-cryptography/round-2-submissions>
- [2] Rupp, A.; Eisenbarth, T.; et al. Hardware SLE solvers: Efficient building blocks for cryptographic and cryptanalytic applications. *Integration*, volume 44, no. 4, 2011: pp. 290–304, ISSN 0167-9260, doi: <https://doi.org/10.1016/j.vlsi.2010.09.001>, hardware Architectures for Algebra, Cryptology and Number Theory. Available from: <https://www.sciencedirect.com/science/article/pii/S016792601000057X>
- [3] Balasubramanian, S.; Bogdanov, A.; et al. Fast Multivariate Signature Generation in Hardware: The Case of Rainbow. In *2008 16th International Symposium on Field-Programmable Custom Computing Machines*, April 2008, pp. 281–282, doi:10.1109/FCCM.2008.52.
- [4] *Rainbow Signature One of the Three NIST Post-quantum Signature Finalists*. [cit. 2021-05-27]. Available from: <https://www.pqcraibow.org/>
- [5] Ding, J. *Rainbow*. Supporting documentation for the NIST contest submission. [cit. 2021-05-13]. Available from: <https://csrc.nist.gov/Projects/post-quantum-cryptography/round-2-submissions>
- [6] Beckman, D.; Chari, A. N.; et al. Efficient networks for quantum factoring. *Phys. Rev. A*, volume 54, Aug 1996: pp. 1034–1063, doi: 10.1103/PhysRevA.54.1034. Available from: <https://link.aps.org/doi/10.1103/PhysRevA.54.1034>
- [7] Biryukov, A.; Khovratovich, D. Related-key Cryptanalysis of the Full AES-192 and AES-256. Cryptology ePrint Archive, Report 2009/317, 2009, <https://eprint.iacr.org/2009/317>.

- [8] Ding, J.; Schmidt, D. Rainbow, a New Multivariable Polynomial Signature Scheme. 06 2005, pp. 164–175, doi:10.1007/11496137_12.
- [9] Courtois, N.; Goubin, L.; et al. Solving Underdefined Systems of Multivariate Quadratic Equations. In *Public Key Cryptography*, edited by D. Naccache; P. Paillier, Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 211–227.
- [10] Frankenfield, J. *Quantum Computing*. Investopedia, [cit. 2022-01-16]. Available from: <https://www.investopedia.com/terms/q/quantum-computing.asp>
- [11] Gambetta, J. *IBM’s Roadmap For Scaling Quantum Technology*. IBM, [cit. 2021-05-13]. Available from: <https://www.ibm.com/blogs/research/2020/09/ibm-quantum-roadmap/>
- [12] Reich, H. *How Quantum Computers Break Encryption — Shor’s Algorithm Explained*. minutephysics, [cit. 2022-01-16]. Available from: <https://www.youtube.com/watch?v=lvTqbM5Dq4Q>
- [13] Zalka, C. Fast versions of Shor’s quantum factoring algorithm. *arXiv: Quantum Physics*, 1998.
- [14] Canadian Centre for Cyber Security. *Cyber Centre’s summary review of final candidates for NIST Post-Quantum Cryptography standards*. [cit. 2021-05-13]. Available from: <https://cyber.gc.ca/en/cyber-centres-summary-review-final-candidates-nist-post-quantum-cryptography-standards>
- [15] NIST. *Post-Quantum Cryptography*. [cit. 2021-05-13]. Available from: [https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/evaluation-criteria/security-\(evaluation-criteria\)](https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/evaluation-criteria/security-(evaluation-criteria))
- [16] Sako, K. *Semantic Security*. Boston, MA: Springer US, 2011, ISBN 978-1-4419-5906-5, pp. 1176–1177, doi:10.1007/978-1-4419-5906-5_23. Available from: https://doi.org/10.1007/978-1-4419-5906-5_23
- [17] Baumslag, G.; Fine, B.; et al. *A Course in Mathematical Cryptography*. De Gruyter, 2015, doi:doi:10.1515/9783110372779. Available from: <https://doi.org/10.1515/9783110372779>
- [18] Kipnis, A.; Hotzvim, H.; et al. Unbalanced Oil and Vinegar Signature Schemes. 01 2000.
- [19] Ding, J.; Deaton, J.; et al. Cryptanalysis of The Lifted Unbalanced Oil Vinegar Signature Scheme. Springer-Verlag, 2020, doi:http://dx.doi.org/10.1007/978-3-030-56877-1_10.

- [20] ARM. AMBA AXI and ACE Protocol Specification. online, 2003, 2004, 2010, 2011, http://www.gstitt.ece.ufl.edu/courses/fall15/ee14720_5721/labs/refs/AXI4_specification.pdf.
- [21] Tang, S.; Yi, H.; et al. High-Speed Hardware Implementation of Rainbow Signature on FPGAs. 11 2011, ISBN 978-3-642-25404-8, pp. 228–243, doi: 10.1007/978-3-642-25405-5_15.
- [22] Ferozpuri, A.; Gaj, K. High-speed FPGA Implementation of the NIST Round 1 Rainbow Signature Scheme. In *2018 International Conference on ReConFigurable Computing and FPGAs (ReConFig)*, Dec 2018, ISSN 2640-0472, pp. 1–8, doi:10.1109/RECONFIG.2018.8641734.